



Midterm Examinations

1st Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Dr. Avigail P. Magbag

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

Syllabus:

- 1. Selection Structures
 - 1.1. The if-else Statement
 - 1.2. Comparing Strings
 - 1.3. Nested Decision Structures
 - 1.4. The if-elif-else Statement
 - 1.5. Structural Pattern Matching (match-case)
- 2. Repetition Structures
 - 2.1. The while loop
 - 2.2. break, continue, and pass functions
 - 2.3. Emergency Exit (sys.exit)
 - 2.4. The for loop

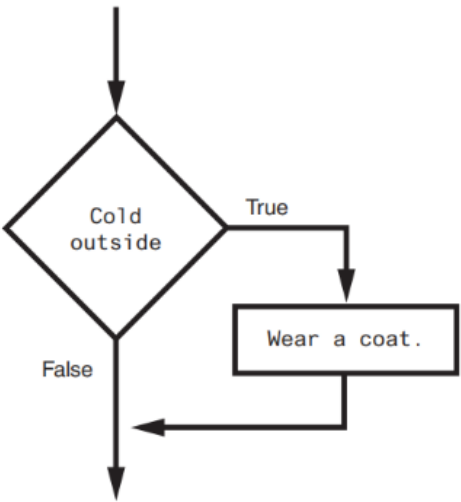
*Should there be any mistakes, inconsistencies, or inaccuracies in this material, please feel free to reach out to the transcriber. Good luck and study well for your exams!

Last updated: September 9, 2024 at 6:48 PM

Selection Structures

- **Control structure** is logical design that controls order in which set of statements execute.
- **Sequence structure** is a type of control structure that consists a set of statements that execute in the order they appear (execution from top to bottom).
- **Decision structure** is a type of control structure in which specific action(s) are performed only if a condition exists (also known as **selection structure**).
- The figure below shows a simple decision structure:

A simple decision structure



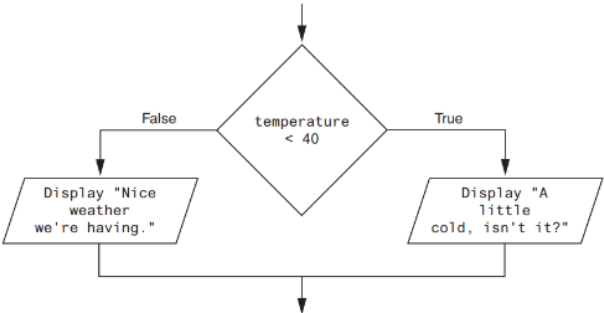
- In the flowchart above, the **diamond symbol** indicates some condition that must be tested. In this case, it needs to be determined whether the condition *Cold* outside is true or false. If this condition is true, the action *Wear a coat* is performed. If the condition is false, the action is skipped.
- The action is conditionally executed because it is performed only when a certain condition is **true**.
- The flowchart above is an example of a **single alternative**, which is a type of a decision structure that provides only one alternative path of execution.
- In Python, the **if** statement is used to write a single alternative decision structure. The syntax or general format is:

```
if condition:
    statement
    statement
    etc.
```

- The first line is known as the **if clause** that begins with the word **if** and followed by a condition (which is an expression that will be evaluated as either true or false). A colon symbol (:) appears after the condition.
- Beginning at the next line is a block of statements. A block is simply a set of statements that belong together as a group. **All of the statements in the block are indented.** This indentation is **required** because the Python interpreter uses it to tell where the block begins and ends.
- When the if statement executes, **the condition is tested**. If the condition is true, the statements that appear in the block following the if clause are executed. If the condition is false, the statements in the block are **skipped**.

The if-else Statement

- An **if-else statement** is a **dual alternative decision structure** has two possible paths of execution - it will execute one block of statements if the condition is true, or another block if the condition is false.
- The figure below is a sample flowchart for a dual alternative decision structure:



- The decision structure in the flowchart tests the condition **temperature < 40**. If this condition is **true**, the message **"A little cold, isn't it?"** is displayed. If the condition is **false**, the statement message **"Nice weather we're having."** is displayed.
- The syntax or general format of the if-else statement is:

```
if condition:
    statement
    statement
    etc.
else:
    statement
    statement
    etc.
```

- When this statement executes, the **condition** is tested.
 - If it is true, the block of indented statements following the **if clause** is **executed**, then control

If the condition is true, this block of statements is executed.

```
if condition:
    statement
    statement
    etc.
else:
    statement
    statement
    etc.
```

Then, control jumps here, to the statement following the if-else statement.



Midterm Examinations

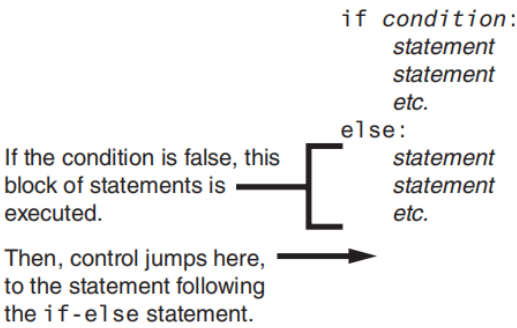
1st Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Dr. Avigail P. Magbag

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

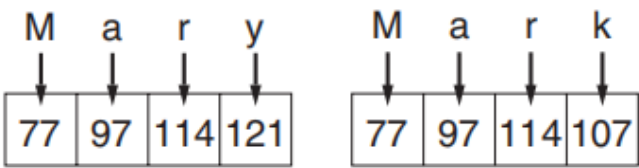
of the program jumps to the statement that follows the if-else statement.

- If the condition is **false**, the block of indented statements following the else clause is executed, then control of the program jumps to the statement that follows the if-else statement.

- Follow these guidelines for the if-else statement indentation:
- if clause and the else clause are **aligned**
 - if clause and the else clause are each followed by a block of statements that are consistently indented



- If a shorter word is substring of longer word, longer word is greater than the shorter word.
- For example, let us compare the two strings **'Mary'** and **'Mark'**:



```
name1 = 'Mary'
name2 = 'Mark'
if name1 > name2:
    print('Mary is greater than Mark')
else:
    print('Mary is not greater than Mark')
```

- The **>** **operator** compares each character in the strings **'Mary'** and **'Mark'**, beginning with the first, or leftmost, characters.
- Here is how the comparison takes place:
1. The 'M' in 'Mary' is compared with the 'M' in 'Mark'. Since these are the same, the next characters are compared.
 2. The 'a' in 'Mary' is compared with the 'a' in 'Mark'. Since these are the same, the next characters are compared.
 3. The 'r' in 'Mary' is compared with the 'r' in 'Mark'. Since these are the same, the next characters are compared.
 4. The 'y' in 'Mary' is compared with the 'k' in 'Mark'. Since these are not the same, the two strings are not equal. The character 'y' has a higher ASCII code (121) than 'k' (107), so it is determined that the string 'Mary' is greater than the string 'Mark'.

Comparing Strings

- In Python, strings are compared by creating decision structures that test the value of a string.
- The **==** operator can be used to compare strings and determine whether they are equal. For example:

```
name1 = 'Mary'
name2 = 'Mark'
if name1 == name2:
    print('The names are the same.')
else:
    print('The names are NOT the same.')
```

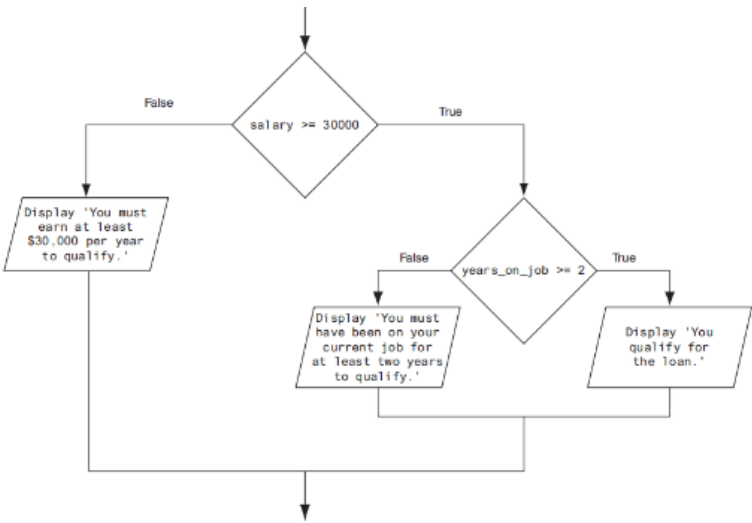
#Output:
'The names are NOT the same.'

- The **!=** operator can be used to compare strings and determine whether they are not equal. For example:

```
if month != 'October':
    print('This is the wrong time for Octoberfest!')
```

- String comparisons are **case sensitive**.
- In addition to determining whether strings are equal or not equal, whether one string is greater than or less than another string can also be determined.
- This can be done using **>**, **<**, **>=**, and **<=** operators.
 - Comparison is character by character based on the ASCII values for each character.
 - The uppercase characters A through Z are represented by the numbers 65 through 90.
 - The lowercase characters a through z are represented by the numbers 97 through 122.
 - When the digits 0 through 9 are stored in memory as characters, they are represented by the numbers 48 through 57.
 - A blank space is represented by the number 32.
 - For example, the string **'abc123'** would be stored in memory as the codes **97, 98, 99, 49, 50, and 51**.

Nested Decision Structures



- To test more than one condition, a decision structure can be nested inside another decision structure.
- The 1st condition **salary >= 30000** is tested. If condition is **false**, there is no need to perform further tests since the customer does not qualify for the loan (customer must earn at least \$30,000/year).
 - If 1st condition is **true**, then the 2nd condition **years_on_the_job >= 2** is tested. If this condition is **true**, then the customer qualifies for the loan. If this condition is **false**, then the



Midterm Examinations

1st Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Dr. Avigail P. Magbag

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

customer does not qualify (must have been employed for at least two years)

- To determine if someone qualifies for a loan, they must meet two conditions: **salary** and **years on the job**

- **Use proper indentation** in a nested decision structure. Not only is proper indentation required by the Python interpreter, but it also makes it easier to see which actions are performed by each part of the structure (improves program code readability).
- Rules for writing nested if statements:
 - else clause should align with matching if clause
 - statements in each block must be consistently indented

The if-elif-else statement

- This is a special version of a decision structure that makes logic of nested decision structures simpler to write.
- General format or syntax:

```
if condition_1:
    statement
    statement
    etc.
elif condition_2:
    statement
    statement
    etc.
#Insert as many elif clauses as
necessary . . .
else:
    statement
    statement
    etc.
```
- When the statement executes, **condition_1** is tested. If **condition_1** is **true**, the block of statements that immediately follow is executed, up to the elif clause. The rest of the structure is ignored.
- If **condition_1** is **false**, however, the program jumps to the very next elif clause and **tests condition_2**. If it is true, the block of statements that immediately follow is executed, up to the next elif clause. The rest of the structure is then ignored.
- This process continues until a condition is found to be true, or no more elif clauses are left. If no condition is true, the block of statements following the else clause is executed.
- Example:

```
if score >= A_SCORE:
    print('Your grade is A.')
elif score >= B_SCORE:
    print('Your grade is B.')
elif score >= C_SCORE:
    print('Your grade is C.')
elif score >= D_SCORE:
    print('Your grade is D.')
else:
    print('Your grade is F.')
```
- Alignment used with if-elif-else statement: **if**, **elif**, and **else** clauses are all aligned while conditionally executed blocks are consistently indented
- The **if-elif-else** statement is never required because its logic can be coded with nested **if-else** statements.
- However, a long series of nested **if-else** statements has two particular disadvantages when debugging the code:

- The code can grow complex and become difficult to understand.
- Because of the required indentation, a long series of nested if-else statements can become too long to be displayed on the computer screen without horizontal scrolling. Also, long statements tend to “wrap around” when printed on paper, making the code even more difficult to read.

Structural Pattern Matching (match-case)

- Until version 3.10, Python did not have a feature for implementing switch statement like other programming languages.
- The if, elif, else statement was utilized to execute the switch statement functionality.
- From version 3.10 onwards, Python has implemented the feature as Structural Pattern Matching implemented using the `match` and `case` keywords.
- General format or syntax:

- ```
match term:
 case pattern1:
 action1
 case pattern2:
 action2
 case pattern3:
 action3
 case _:
 default action
```
- The **term** refers to the variable that holds the value to match with any of the cases the follow.
  - The underscore symbol (`case _:`) is used to define a default case to execute when no match is found. This is similar to the `else` statement in if-elif-else structures.
  - Below is an example of an if-elif-else structure with its corresponding `match-case` structure. Both yield the same output and functionality.

```
cType = input("Customer Type [R or C:] ")

if cType == 'R' or cType == 'r':
 gen_rate = 4.50
 evat_rate = 0.13
elif cType == 'C' or cType == 'c':
 gen_rate = 3.20
 evat_rate = 0.15
else:
 print ("Invalid customer type.")
 KWH = 0

cType = input("Customer Type [R or C:] ")
match cType:
 case 'R' | 'r':
 gen_rate = 4.50
 evat_rate = 0.13
 case 'C' | 'c':
 gen_rate = 3.20
 evat_rate = 0.15
 case _:
 print ("Invalid customer type.")
 KWH = 0
```



# Midterm Examinations

1st Semester, S.Y. 2024 – 2025  
Transcribed by: Aerith Claude Catap  
Instructor: Dr. Avigail P. Magbag

HAU School of Computing  
First Year | Bachelor of Science in  
Computer Science

## Repetition Structures

- Often times, you would have to write code that performs the same task multiple times. However, there comes disadvantages to duplicating blocks of code:
  - Disadvantages to duplicating code
  - Makes program large
  - Time consuming
  - May need to be corrected in many places
- A **repetition structure** makes a computer repeat included code as necessary
- Includes **condition-controlled loops** and **count-controlled loops**

## The while loop

- It is also known as a **pretest loop**.
- Often used for **condition-controlled** loops, as it continues the loop as long as a certain condition is **true**.
- Its structure generally consists of two parts:
  - A condition for true or false value
  - Block of statements repeated as long as condition is true
- In order for a *while* loop to stop executing, something has to happen inside the loop to make the condition false to terminate it. Otherwise, the loop will run infinitely.
- An **iteration** refers to one execution of the body of a loop.
- General format or syntax:

```
while condition1:
 statement1
 statement2
 etc.
```

Example:

```
1 keep_going = 'y'
2
3 while keep_going == 'y':
4 sales = float(input("Enter the amount of sales: "))
5 comm_rate = float(input("Enter the commission rate: "))
6 commission = sales*comm_rate
7 print ("The commission is $", format(commission, '.2f'), sep = '')
8 keep_going = input ("Do you want to calculate another commission? (Enter y for yes): ")
```

- In this code, the condition “while keep\_going == 'y' :” has a Boolean value of True. Hence, the loop is initiated and the statements inside the loop will be repeated continuously (until the condition is set to false).
- After the first iteration of the loop, the condition will be re-evaluated. If it turns out to be false, then the loop is terminated
- Inside the loop, the statement “keep\_going = input ("Do you want to calculate another commission? (Enter y for yes): ")” can terminate the loop. If the user inputs anything other than y, the condition will evaluate to false, thereby ending the loop.

## break, continue, and pass functions

- a. **break** is used in conjunction with conditional statements (if-elif-else) to terminate the loop early if some condition is met. Specifically, the break statement provides a way to exit the loop entirely before the iteration is over. Example:

```
for i in range(5):
 if i == 3:
 break
```

```
print(i)
print ('Finished')
```

#Output:  
0  
1  
2  
Finished

- b. Once the condition in the second line evaluates to True, the **continue** statement skips over the print statement inside the loop. It then continues the loop at the next element. The loop ends when the last element is reached. Example:

```
for i in range(5):
 if i == 3:
 continue
 print(i)
print ('Finished')
```

#Output:  
0  
1  
2  
4  
Finished

- c. Strictly speaking, **pass** isn't a way to exit a loop in Python. You'll notice all elements defined by range() are printed to the screen. The pass statement serves as a placeholder for code you may want to add later in its place.

```
for i in range(5):
 if i == 3:
 pass
 print(i)
print ('Finished')
```

#Output:  
0  
1  
2  
3  
4  
Finished

## Emergency Exit in Python

- If you want to exit a program completely before you reach the end, the **sys module** provides that functionality with the **exit() function**. Calling this function raises a **SystemExit exception** and terminates the whole program.
- **sys.exit()** accepts one optional argument. It may be either an integer or a string, which may be used to print an error message to the screen. Example:

```
for i in range(5):
 if i == 3:
 sys.exit("Error message")
 print(i)
print ('Finished')
```

#Output:  
0  
1  
2  
SystemExit: Error Message





# Midterm Examinations

1st Semester, S.Y. 2024 – 2025  
Transcribed by: Aerith Claude Catap  
Instructor: Dr. Avigail P. Magbag

HAU School of Computing  
First Year | Bachelor of Science in  
Computer Science

- In this context, `sys.exit()` behaves similarly to break in the earlier example, but also raises an exception. As a result, the final print statement outside the `for` loop is not executed in this example. The whole program is simply terminated.

## The for loop

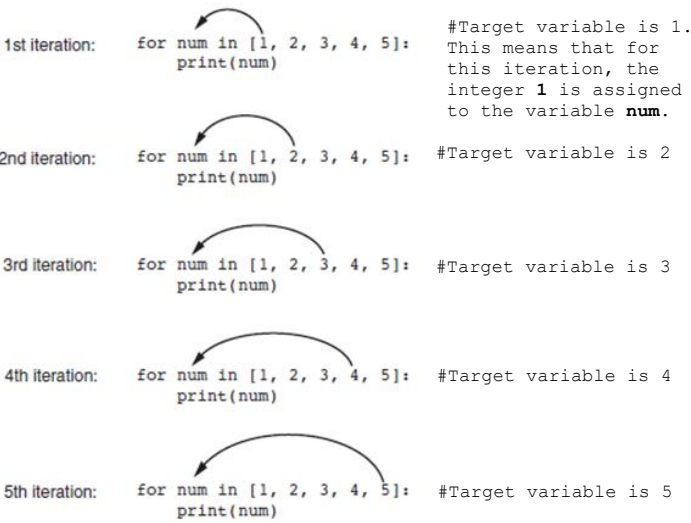
- Often used for **count-controlled** loops, as it iterates the loop for defined counts.
- It is designed to work with sequence of data items
- General format or syntax:

```
for variable in [val1, val2, etc.]:
 statements
```

#The range function can also be used in conjunction with the for loop.

```
for variable in range (value):
 statements
```

- **Target variable** – the variable which is the target of the assignment at the beginning of each iteration.
  - The purpose of target variable is to reference each item in a sequence as the loop iterates
  - Target variable can be used in calculations or tasks in the body of the loop



## Using the range Function with the for Loop

- The range function simplifies the process of writing a for loop.
  - `range` returns an **iterable** object, meaning it contains a sequence of values that can be iterated over
- range characteristics:
  - One argument: used as ending limit. Example:

```
for i in range(50):
 print(i)
```

    - The loop will iterate 50 times. On the first iteration, the value assigned to `i` is **0, as this is the default starting value** for range functions.
    - The loop will end with one value before 50, which is 49. Hence, the last value assigned to `i` is 49.
    - Therefore, this code will print all values beginning from 0 to 49.

- Two arguments: starting value and ending limit.  
Example:

```
for i in range(5,50):
 print(i)
```

- This time, the first argument in the range function, which is 5, is the starting value of the range. So, the value assigned to `i` on the first iteration is 5.
- The loop will end with one value before the ending limit, which is 49. Hence, the last value assigned to `i` is 49.
- Therefore, this code will print all values beginning from 5 to 49.

- Three arguments: third argument is step value  
Example:

```
for i in range(0,20,4):
 print(i)
```

- Similarly, the first argument in the range is the starting value, which is 0. So, the value assigned to `i` on the first iteration is 0.
- The loop will still end with one value before the ending limit, which is 19.
- The third argument in the range, which is 3, is the step value. This determines the interval by which the range increments in each iteration.
- Therefore, this code will produce this output:  
0  
4  
8  
12  
16

- Sometimes, the programmer does not know exactly how many times the loop will execute
- You can receive range inputs from the user, place them in variables, and call the range function in the `for` clause using these variables
- Be sure to consider the end cases: range does not include the ending limit. Example:

```
start = int(input("Start value: "))
end = int(input("End value: "))
step = int(input("Step value: "))
```

```
for i in range(start, end+1, step):
 print (i)
```

#User inputs:  
Start value: 1  
End value: 15  
Step value: 2

#Output:  
1  
3  
5  
7  
9  
11  
13  
15